



EC-200 Data Structures

LAB MANUAL # 10

Course Instructor: Dr. Anum Abdul Salam

Lab Engineer: Engineer Hira Irshad

Student Name: _____

Degree/ Syndicate: _____

	Trait	Obtained Marks	Maximum Marks
R1	Application Functionality 20%		20
R2	Specification & Data structure implementation 30%		30
R3	Reusability 10%		10
R4	Input Validation 10%		10
R5	Efficiency 20%		20
R6	Delivery 10%		10
R7	Plagiarism above 60%		0
Total			100

Total Marks = *Obtained Marks* ($\sum_1^6 R_i * R_7$)

Lab # 10: Sorting Algorithms

Lab Objective:

To implement sorting algorithms.

Lab Description:

1. Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list. The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right. Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



The same process is applied to the rest of the items in the array.

2. Bubble Sort

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. We take an unsorted array for our example.



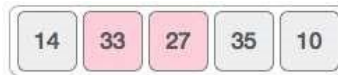
Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this



Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this



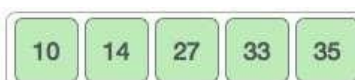
To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this



Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sorts learns that an array is completely sorted.



3. Insertion Sort

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element which is to be 'insert'ed in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence named insertion sort. The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array).

4. **Merge Sort:**

Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being $(n \log n)$, it is one of the most used and approached algorithms.

Merge sort first divides the array into equal halves and then combines them in a sorted manner.

To understand merge sort, we take an unsorted array as the following –



We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.



We further divide these arrays and we achieve atomic value which can no more be divided.



Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of four data values placing all in a sorted order.



After the final merging, the list becomes sorted and is considered the final solution.



5. **Quick Sort:**

Quick sort is a highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than the specified value, say pivot, based on which the partition is made and another array holds values greater than the pivot value.

Quicksort partitions an array and then calls itself recursively twice to sort the two resulting subarrays. This algorithm is quite efficient for large-sized data sets as its average and worst-case complexity are $O(n^2)$, respectively.

Lab Task:

Implement the following sorting algorithms in C++:

- Selection Sort
- Bubble Sort
- Insertion Sort
- Merge Sort
- Quick Sort

Your program should take an array of integers as input, apply each sorting algorithm separately, and display the sorted output.

